

The figure illustrates the architectural design of discriminators D_C and D_T , presented as a sequential, left-to-right flow of convolutional layers. The global layout is linear and horizontal, depicting a feed-forward network structure with five main stages, each composed of a repeating block of three components: a 4×4 convolution layer, followed by spectral normalization, and then a LeakyReLU activation. Each stage is visually represented by a group of three rectangular blocks arranged side-by-side, with the entire sequence progressing from left to right. Each block within a stage has distinct visual attributes: the 4×4 convolution is shown as a light gray rectangular prism; spectral normalization is depicted as a medium blue rectangular prism; and LeakyReLU is represented as a light orange rectangular prism. These color-coded blocks are consistently used throughout the diagram to denote the same operations. Below each stage, the output dimensions are specified in mathematical notation, indicating spatial resolution and channel count. For instance, the first stage outputs $W/2 \times H/2 \times 64$, the second $W/4 \times H/4 \times 128$, the third $W/8 \times H/8 \times 256$, the fourth $(W/8-1) \times (H/8-1) \times 512$, and the final stage $(W/8-2) \times (H/8-2) \times 1$. The connections between stages are indicated by arrows pointing from the right end of one stage to the left end of the next, signifying the data flow. Above each stage, the stride value is specified: Stride = 2 for the first three stages and Stride = 1 for the last two. A legend at the bottom clarifies the meaning of each colored block: light gray corresponds to 4×4 Convolution, medium blue to Spectral Normalization, and light orange to LeakyReLU. The caption specifies that all convolutions use a kernel size of 4 and padding of 1, and that LeakyReLU uses a slope of 0.2. The overall structure reflects a progressive feature extraction process, reducing spatial dimensions while increasing channel depth, culminating in a single scalar output, likely for binary classification.

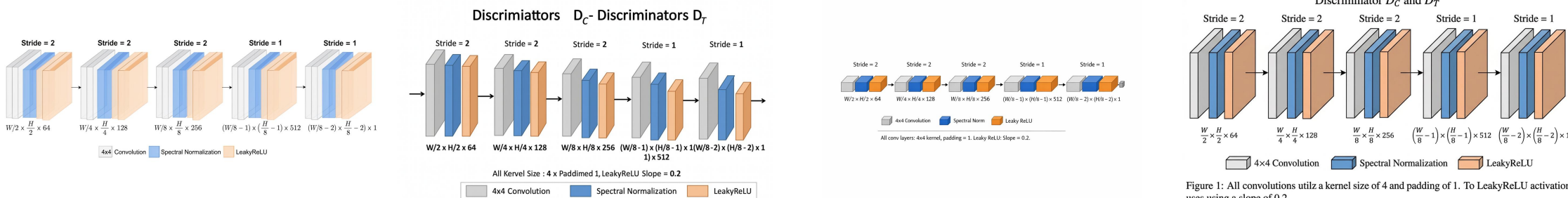
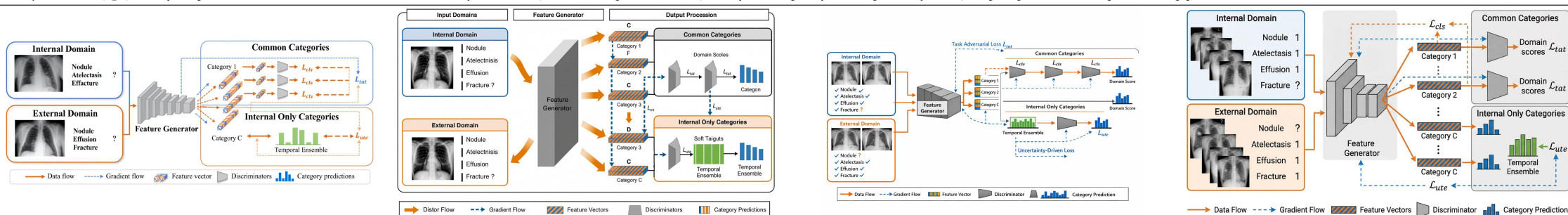
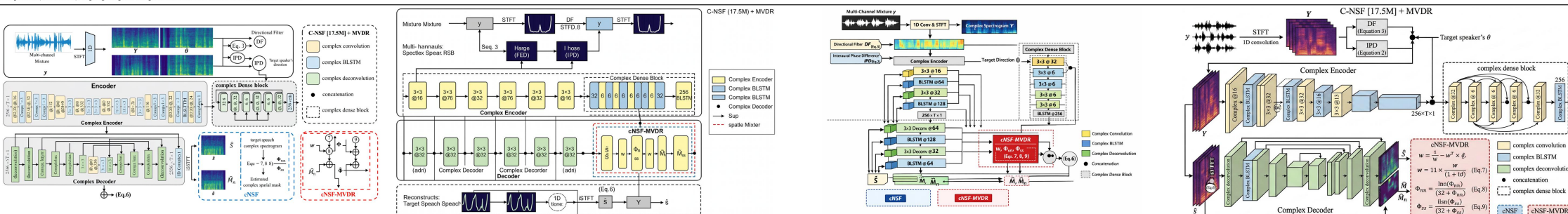


Figure 1: All convolutions utilize a kernel size of 4 and padding of 1. To LeakyReLU activation uses a slope of 0.2.

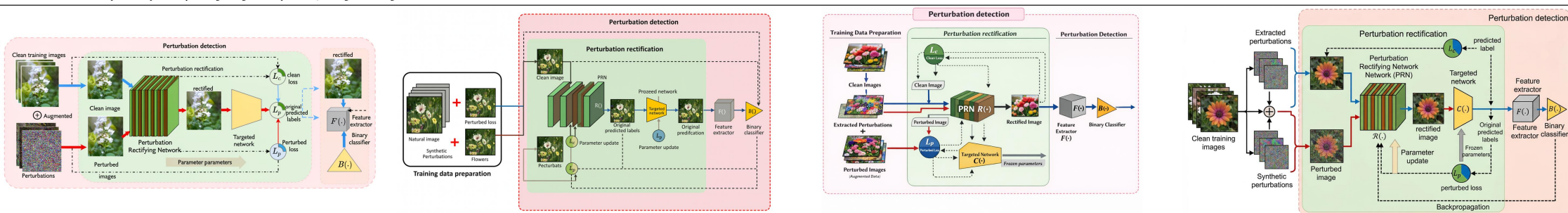
The figure presents an overview of a multi-domain medical image classification framework designed to handle data from two distinct sources—Internal Domain and External Domain—with differing visual appearances and partially overlapping label sets. The global layout is horizontally structured into three main sections: input domains on the left, a central Feature Generator module, and output processing blocks on the right, divided into Common Categories and Internal Only Categories. A legend at the bottom clarifies visual elements: solid orange arrows denote data flow, dashed blue arrows indicate gradient flow, orange-and-blue striped bars represent feature vectors, gray trapezoids symbolize discriminators, and blue bar graphs signify category predictions. On the left, two input boxes represent the Internal Domain (blue border) and External Domain (orange border), each containing chest X-ray images and associated binary labels for conditions such as Nodule, Atelectasis, Effusion, and Fracture. The Internal Domain has a question mark next to 'Fracture', while the External Domain has a question mark next to 'Fracture'. The Feature Generator (a gray 3D block) receives inputs from both domains. The output processing blocks on the right are divided into Common Categories (Category 1, 2, 3) and Internal Only Categories (Category C). The Common Categories section shows domain scores for each category, with a task adversarial loss L_{task} and a domain score L_{cls} . The Internal Only Categories section shows a temporal ensemble of predictions, with a loss L_{ute} and a temporal ensemble of predictions. The framework is trained using a task-specific adversarial loss, denoted by L_{task} , with gradient flow (dashed blue arrows) and data flow (solid orange arrows) indicated. The loss is calculated for the Internal Domain, External Domain, and Common Categories. The framework is trained using a task-specific adversarial loss, denoted by L_{task} , with gradient flow (dashed blue arrows) and data flow (solid orange arrows) indicated. The loss is calculated for the Internal Domain, External Domain, and Common Categories.



The figure illustrates the architecture of a complex neural spatial filter (cNSF) model for target speech separation (TSS), combined with MVDR (Minimum Variance Distortionless Response) processing, denoted as 'cNSF [17.5M] + MVDR'. The overall layout is a deep learning pipeline structured into three main horizontal sections: input processing, a central encoder-decoder network, and output reconstruction. The top section shows the input multi-channel mixture signal y , represented as waveforms, which undergoes Short-Time Fourier Transform (STFT) via a 1D convolution layer to produce complex spectrograms Y . These spectrograms are processed in parallel to compute Directional Filter (DF) using Equation 3 and Internal Phase Difference (IPD) using Equation 2. The computed DF and IPD, along with the target speaker's direction θ , are fed into the Complex Encoder. The Complex Encoder consists of a sequence of alternating complex convolution layers (light yellow blocks) and complex BLSTM layers (light blue blocks), forming an encoder path that progressively reduces spatial dimensions while increasing feature depth. Each convolution layer is specified by kernel size (e.g., 3×3) and number of filters (e.g., $@16, @32$, etc.), with the final encoded representation having dimensions $256 \times T \times 1$. This is followed by a complex dense block (dashed box on the right), composed of five complex convolution layers with varying filter counts (32, 6, 6, 6, 32) and skip connections between them, culminating in a 256-cell complex BLSTM layer. The decoder path, labeled 'Complex Decoder', mirrors the encoder but uses complex deconvolution layers (light green blocks) to upsample features back to the original spectrogram dimensions. The decoder receives skip connections from corresponding encoder layers, enabling feature reuse. The decoder outputs two streams: one for the estimated complex spectrogram of target speech $\hat{S}$ and another for the estimated complex spatial mask $\hat{M}$ (for target speech) and $\hat{M}_n$ (for noise). These are computed within a red-dashed box labeled 'cNSF-MVDR', involving computations defined by Equations 7, 8, and 9, which derive weights w and spatial masks Φ_n and Φ_{ss} . The final stage reconstructs the estimated target speech signal $\hat{s}$ via inverse STFT (iSTFT), implemented as a 1D convolution, using the estimated spectrogram $\hat{S}$ and the original mixture Y . A circular symbol labeled (Eq.6) indicates the combination operation for this reconstruction. The figure includes a legend on the right defining visual elements: light yellow for complex convolution, light blue for complex BLSTM, light green for complex deconvolution, black dots for concatenation, and dashed boxes for complex dense blocks. Two dashed boxes at the bottom right distinguish the cNSF component (blue dashed) from the cNSF-MVDR component (red dashed), highlighting the integration of MVDR within the neural framework.



The figure illustrates the architecture and training pipeline of the Perturbation Rectifying Network (PRN), designed for detecting and mitigating adversarial perturbations in images. The global layout is divided into two main stages: training data preparation on the left and the core PRN framework with perturbation detection on the right. The entire process is enclosed within a light pink dashed boundary labeled 'Perturbation detection'. On the left side, clean training images are shown as a stack of natural images (e.g., flowers). These images are augmented with synthetic perturbations generated from extracted perturbations—represented as noisy, colorful patches—which are added via a circular plus symbol. This augmentation produces perturbed images used for training. The central part of the diagram, highlighted with a light green background and labeled 'Perturbation rectification', contains the core components of PRN. A clean image and a perturbed image are both fed into the Perturbation Rectifying Network (PRN), depicted as a 3D block with alternating green and brown vertical stripes, symbolizing multiple layers or filters. The PRN outputs a rectified image, which is then passed through a 'Targeted network'—shown as a yellow trapezoid—representing a pre-trained classifier whose parameters are frozen during training. The targeted network produces original predicted labels. Two loss functions are computed: L_c (clean loss) and L_p (perturbed loss), represented by circular nodes with green and blue segments. L_c compares the predicted label from the clean image with the rectified image, while L_p compares the predicted label from the perturbed image with the rectified image. Both losses are connected via dashed lines to the PRN, indicating backpropagation for parameter updates. A beige arrow labeled 'Parameter update' points from the losses to the PRN, while a gray arrow labeled 'Frozen parameters' points from the PRN to the Targeted network. On the right side, outside the green box but still within the pink boundary, the rectified image is further processed by a 'Feature extractor' (a gray box labeled $F(\cdot)$) followed by a 'Binary classifier' (a yellow triangle labeled $B(\cdot)$), forming the perturbation detection module. Solid blue arrows indicate the forward flow from the PRN output to this detection branch. Dashed blue arrows represent feedback loops for loss computation and backpropagation. The diagram uses color-coded arrows: blue for clean image paths, red for perturbed image paths, and black dashed lines for loss connections and backpropagation. Text annotations clarify each component's role, including mathematical notations like $R(\cdot)$ for the PRN function, $C(\cdot)$ for the targeted network, and $B(\cdot)$ for binary classification. The overall workflow emphasizes training PRN to reconstruct clean features from perturbed inputs while preserving the target model's predictions, enabling robustness against adversarial attacks.



The figure presents a comparative schematic of two deep learning models—Model 1: Graph Kernel Network (GKN) and Model 2: Vector Cloud Neural Network (VCNN)—designed for processing point clouds while ensuring frame invariance. The overall layout is divided into two main horizontal sections, each enclosed in a dashed boundary: the upper blue-dashed box for GKN and the lower red-dashed box for VCNN. Both models receive the same input from the left, labeled as (a) Input Matrix of Cloud, which consists of a point cloud represented as a matrix $Q = [q_1^T, q_2^T, \dots, q_n^T]$, where each vector $q_i = [x_i^T, y_i^T, z_i^T]^T$ contains coordinates x_i, y_i, z_i and scalar features c_i . A gray oval illustrates the cloud with points and a highlighted star, indicating a reference point or centroid. For Model 1 (GKN), the workflow begins with (b1) Invariant Edge Features, shown in a green box, depicting a fully connected graph with nodes q_i and edges e_{ij} , emphasizing rotational invariance through edge features. This feeds into (c1) Graph Representation, a gray box containing a kernel network structure with neurons, processing edge features. The output is (d1) Global Averaging, a yellow box showing the formula $\tau = F(\frac{1}{n} \sum_{i=1}^n z_i^T)$. For Model 2 (VCNN), the workflow begins with (b2) Pairwise Projection, shown in a green box, depicting a fully connected graph with nodes q_i and edges e_{ij} , emphasizing rotational invariance through pairwise projections. This feeds into (c2) Projection and Averaging, a gray box containing a kernel network structure with neurons, processing pairwise projections. The output is (d2) Fitting Network, a yellow box showing the formula $\tau = F(\frac{1}{n} \sum_{i=1}^n z_i^T)$. Both models output a global descriptor τ , visualized as a gray oval with scattered points and a central star, symbolizing the learned representation. The outputs are connected back to the original cloud via blue (for GKN) and red (for VCNN) arrows, indicating the flow of information. Color coding is consistent: green boxes denote components ensuring rotational invariance, yellow boxes denote those ensuring permutation invariance, and gray boxes represent general processing modules. The caption clarifies that translational invariance is achieved via relative coordinates in the input, and nonlocal mapping is enabled by the kernel network in GKN and the fitting network in VCNN.

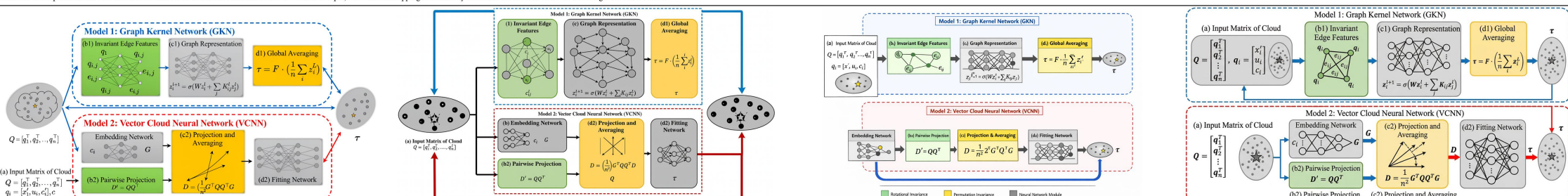


Figure 1: Translational invariance is handled by relative coordinates in the input, and nonlocal mapping is facilitated by the graph kernel in GKN and the fitting network in VCNN.

The figure presents an overview of a self-paced learning (SPL) framework for facial expression recognition, structured into three main components: (a) SPL, (b) CNN, and (c) NDFs, arranged horizontally from left to right. The global layout begins with the SPL module on the far left, followed by the CNN backbone in the center, and concludes with the NDFs module on the right, with feedback connections leading back to the loss computation at the top. In section (a) SPL, a grid of face images is shown, organized by emotion labels such as 'Sad', 'Fear', and 'Disgust'. Each category contains multiple pairs of target and reference images. A red rectangular box highlights a subset of these images labeled as 'Easy samples', indicating that only these selected samples are fed into the subsequent network stages during training. These easy samples are represented as small square images, with arrows pointing from them to the CNN module. Section (b) CNN illustrates the backbone convolutional neural network. It consists of a sequence of 3D blocks representing feature maps, alternating with 'Normalize' blocks. The first block is a 3D orange-and-gray 3D structure, followed by a 'Normalize' block, and so on with ellipses indicating additional layers. The final feature map is processed through another 'Normalize' block before being passed to the NDFs module. The feature map is depicted as a 3D orange-gray block. Section (c) NDFs details the neutral expression-disentangled features computation. The normalized feature map from the CNN is fed into an 'Avg Pool' operation, which outputs two vertical feature vectors: V_{tar} (orange) and V_{ref} (gray). These vectors are then input into a subtraction operation (represented by a circle with a minus sign) to produce a feature vector V_{exp} . This V_{exp} is then passed to a fully connected (FC) layer, which outputs a column of gray circles, each representing a logit value. The output of Softmax is connected to a 'CE Loss' block, which in turn provides feedback to the SPL module, completing the training loop. The legend at the bottom clarifies visual elements: a red square denotes 'Easy samples'; a 3D orange-gray block represents 'Feature map'; orange and gray bars represent 'Feature vector'; a circle with a minus sign indicates 'Subtraction'; and gray circles denote 'Logit value'. All modules are enclosed in dashed-line boxes with corresponding labels (a), (b), and (c). Arrows indicate data flow: from SPL to CNN, from CNN to NDFs, from NDFs to Softmax and CE Loss, and from CE Loss back to SPL. The overall workflow reflects a self-paced learning strategy where easy samples are selected, processed through a normalized CNN, and disentangled into expression-specific features via subtraction, with loss feedback guiding sample selection in future iterations.

